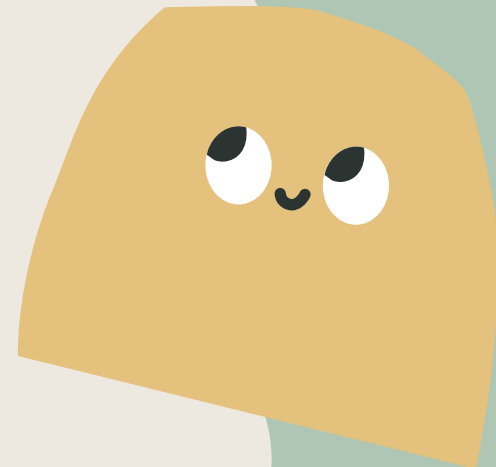
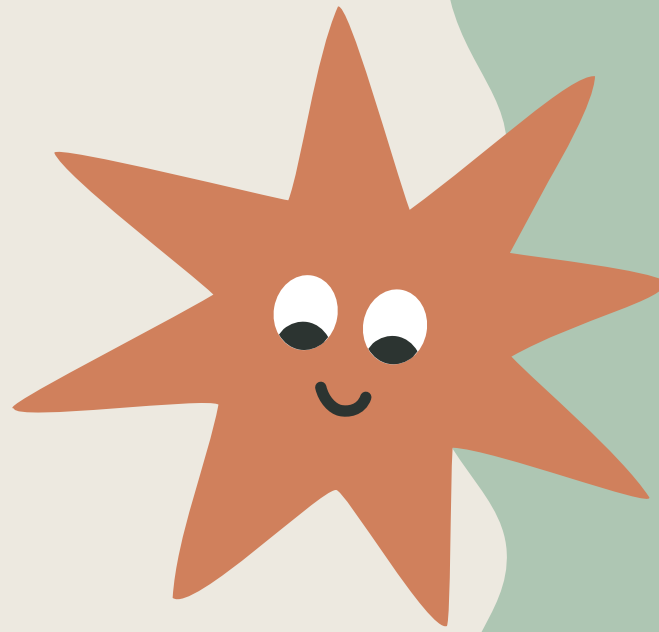


# Introduction to Data Structures and Algorithms

Lecture 1



# Learning Outcomes (Lecture 1)

- At the end of this lecture, students will be able to:
  - Understand the purpose of the Data Structures & Algorithms course.
  - Explain what a data structure is.
  - Understand why data structures are essential in software development.
  - Understand how computer memory works.
  - Differentiate between static and dynamic memory allocation.
  - Recognize real-world applications of different data structures.

# Pre-Requisite for This Course

- Before taking this course, you should know
- Programming Fundamentals
  - Variables, Data Types, Decision Making,
  - Loops, Functions, Arrays
- Object Oriented Programming
  - Classes, Objects, Constructors, Member Functions
  - Access Specifiers, Basic Pointers
- Basic Mathematics
  - Basic Algebra, Logarithms (Introductory Idea)
  - Powers, Summations

# Course Introduction

- Suppose you are developing
  - Facebook
  - WhatsApp
  - YouTube
  - Google Maps
  - PUBG
- What is common among all of them?
- Every software stores, organizes, searches and processes huge amounts of data.
- This is exactly why Data Structures exist

# Why Study Data Structures?

Without Data Structures,

- Google Search would be slow.
- WhatsApp couldn't instantly find chats.
- Facebook couldn't recommend friends.
- YouTube couldn't recommend videos.
- Banking software would become inefficient.

# Definition

- A data structure is a particular way of organizing data in computer memory so that it can be used efficiently
- Algorithm is a step-by-step procedure to solve a problem
- Knowledge about Data Structures and Algorithms (DSA) help us to use large amount of data to solve problem efficiently

## Example 1

- Imagine your wardrobe.
  - If clothes are thrown randomly
  - Finding one shirt takes several minutes.
- If clothes are organized
- Shirts together
- Pants together
- Shoes together
  - Finding becomes much easier.
- That organization is exactly what a Data Structure does.

## Example 2

- Library contains thousands of books.
- Can you imagine books placed randomly?
- Impossible.
- Books are organized
  - Subject
  - Author
  - ISBN
- That organization is a Data Structure.

# Classification of Data Structure

## 1. Primitive Data Structure

- Primitive data structures are the **basic built-in data types** provided by a programming language.
- They can store only **one value** at a time e.g. int, char, float, double etc
- They are primitive because they are the **building blocks** of all other data structures

## 2. Non-Primitive Data Structure

- Non-primitive data structures are created using primitive data types
- They can store **multiple values** and organize data efficiently e.g. arrays, linked lists, stacks, queues, trees, graphs etc

# Non-Primitive Classification

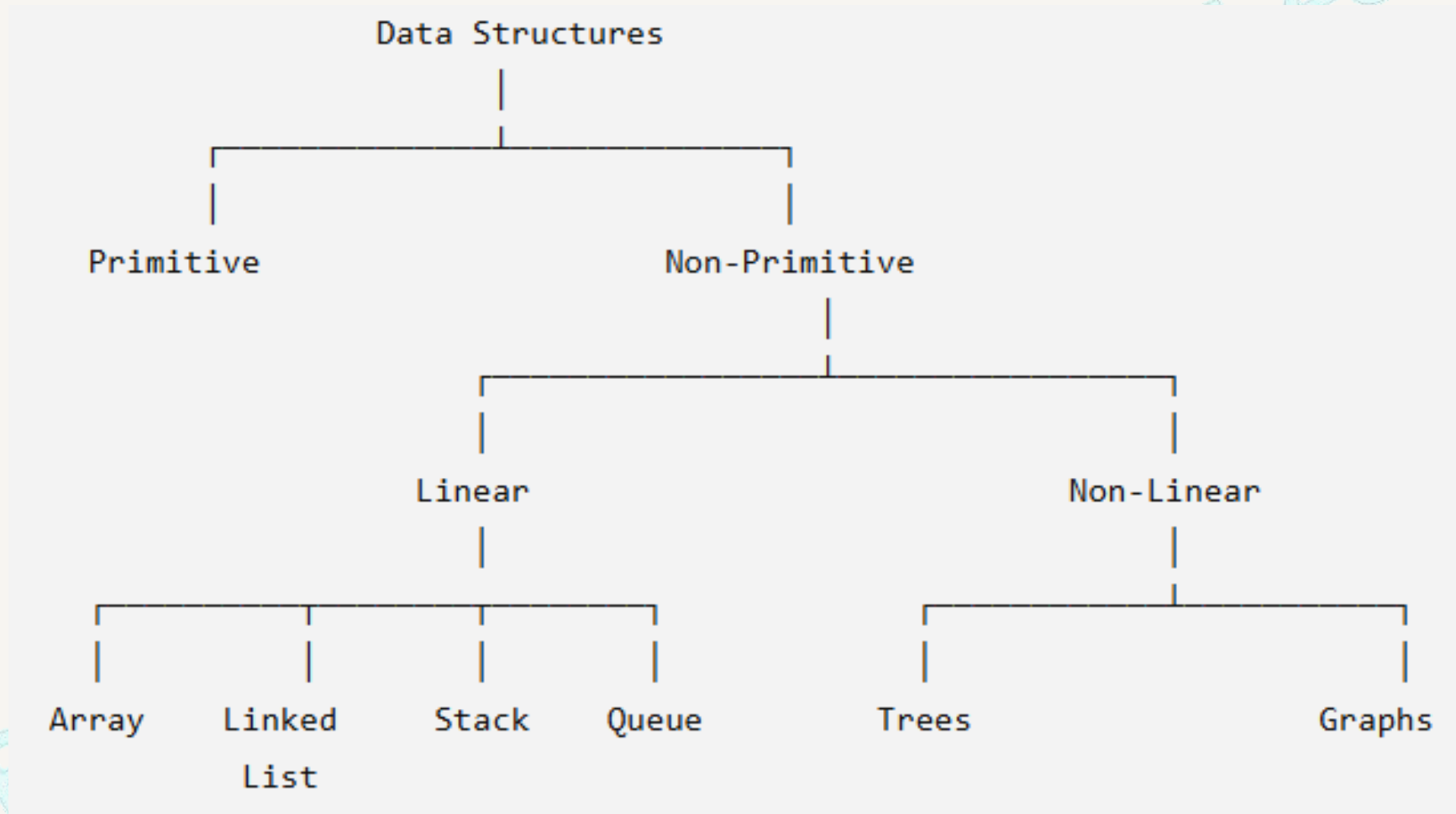
## 1. Linear Data Structure

- A Linear Data Structure stores data **in a sequence**
- Each element has one predecessor and one successor (except the first and last)
- Examples include array, linked list, stack, queue etc

## 2. Non-Linear Data Structure

- In non-linear data structure elements are not organized in a **sequential fashion**
- A data item can be attached to several other data elements
- Examples include trees and graphs

# Classification of Data Structure



# Characteristics of Good Data Structures

Good data structures should

- Store data efficiently
- Use less memory
- Allow fast searching
- Allow easy insertion
- Allow easy deletion
- Scale to millions of records

# Real Examples

## ➤ Google

- Uses graphs, hash tables, trees

## ➤ Facebook

- Uses graphs for friend recommendation

## ➤ WhatsApp

- Uses queues (messages arrive in order)

## ➤ Browser

- Uses stack (back button)

## ➤ GPS

- Uses graphs to find shortest path

# Industry Perspective

## ➤ Companies hiring DSA skills

- Google
- Microsoft
- Amazon
- Meta
- Apple
- Netflix

## ➤ Almost every coding interview includes Data Structures and Algorithms

# Memory Organization

- Where are variables stored?
- RAM
- Every variable has address and value
- Program memory layout
- Code
  - Stores instructions
- Stack
  - Stores local variables
- Heap
  - Stores dynamic memory

## MAIN MEMORY (RAM)

Low Address

0x00000000

⋮

0x00400000

0x10000000

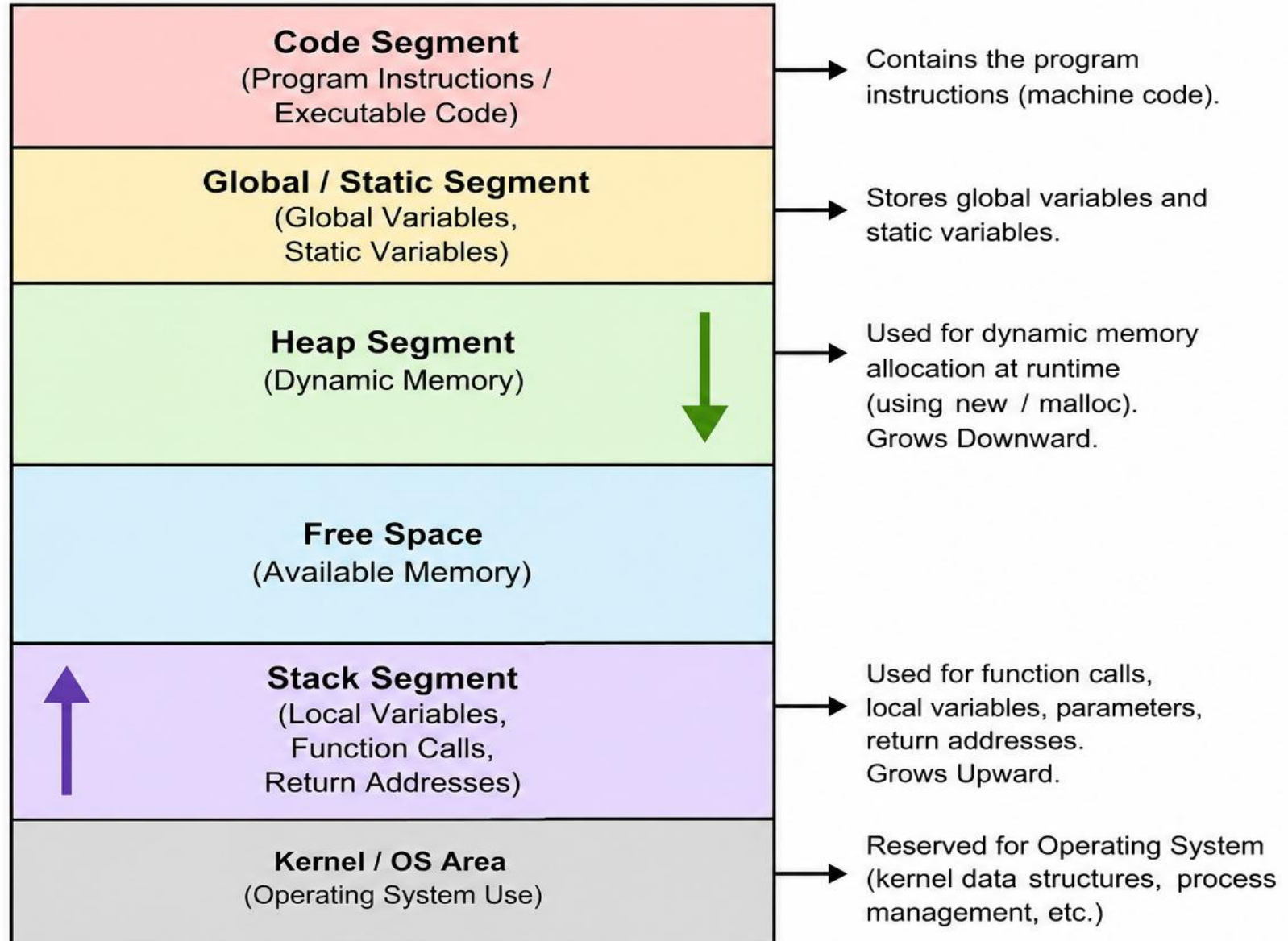
0x7FFFFFFF

0x80000000

⋮

High Address

0xFFFFFFFF



# Static/Stack Memory

➤ Memory allocated before program execution is called Static or Stack Memory

➤ Characteristics

- Fast
- Automatic
- Small
- Managed by compiler

➤ Example

```
void fun()
{
    int a=10;
}
```

➤ Variable disappears after function ends

# Dynamic/Heap Memory

➤ Memory allocated during execution is called Dynamic or Heap Memory

➤ Characteristics

- Large
- Dynamic
- Managed by programmer

➤ Example

```
int *arr=new int[1000];
```

➤ Memory remains until deleted

➤ Deallocation

```
delete[] arr;
```

- Heap memory becomes free
- Pointer disappears automatically when function ends

The pointer **arr** is on the **Stack**.

The actual array is on the **Heap**.

This is a very important concept.

# Memory Leak

➤ Suppose

```
void test()
{
    int *arr = new int[1000];
}
```

➤ Function ends, pointer disappears

➤ But heap memory still exists

➤ Nobody can access it now

➤ This unused allocated memory is called **Memory Leak**.

“A **memory leak** occurs when a program allocates memory from the **Heap** but never releases it after it is no longer needed”

# Drawbacks of Memory Leak

## 1. Wastage of RAM

- Memory remains occupied even though the program is no longer using it
- If a program allocates 100 MB and only use 10 MB, but forget to release 90MB.
- That 90 MB becomes unavailable

## 2. Reduced System Performance

- As free memory decreases, the Operating System spends more time
  - managing memory
  - swapping data
  - allocating new memory
- Result
  - Program becomes slower
  - Entire computer may slow down

# Drawbacks of Memory Leak

## 3. Program Crash

- Suppose a loop continuously allocates dynamic memory
- Memory leaks increasing eventually RAM full and program may crash

## 4. Multiple Programs are Affected

- RAM is shared. Suppose a computer has 8 GB of RAM
- One application leaks 5 GB now only 3 GB remains for different programs like Chrome, Visual Studio, MS Word, Windows
- Everything becomes slow

# Drawbacks of Memory Leak

## 5. Server Downtime

- This is the biggest real-world problem
- Imagine a banking server runs 24 hours, 7 days, 365 days
- Every request leaks 5 KB after 1,000,000 users memory leaks becomes 2 GB
- Eventually banking server crashes and millions of users cannot access their account

## 6. Security Issues

- Memory leaks may expose sensitive information e.g. heap memory may still contain passwords, credit card numbers, personal information
- Attackers may exploit poorly managed memory in some situations